

Rethinking YANG-based Service and Network Management in the Era of Agentic AI

An investigation into how the integration of agents into network management operations impacts the traditional service, network, and device YANG models.

draft-mw-nmop-yang-ai-challenges

Outline

- **Background & Goals**

Rethinking YANG when the industry shifts towards Agentic AI

- **Two workflow Examples**

Share representative workflow examples in agentic architecture

- **Core Challenges Analysis**

Deep dive into major gaps impeding the effective AI-YANG integration

- **Potential Ways Forward**

Exploring technical and architectural pathways to bridge the identified gaps

- **Open Discussion**

Interactive Q&A, feedback sharing, and collaborative idea exchange

Background and Goals

Background

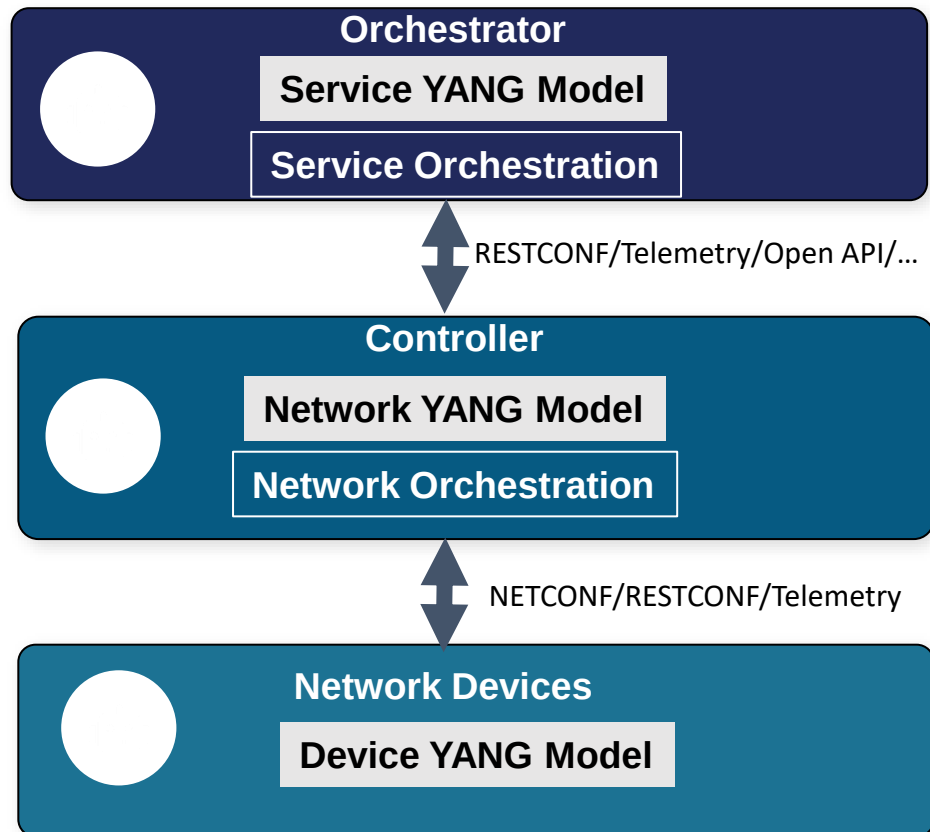
- Agentic AI introduces new paradigms for network automation
- Most existing IETF efforts focus on interface definitions for agents; there is a gap in understanding how the Agentic AI would impact YANG.
- What we are targeting here is the ultimate state of zero-touch/human-out-of-loop AI driven autonomy, not AI-assisted OPS.

Primary Goals

- Facilitate community discussion
- Identify key gaps (if any) that emerge when AI agents operate on YANG
- Explore complementary directions that potentially bridge those gaps

The RFC 8969 Paradigm for Model-driven Automation

RFC8969: A Framework for Automating Service and Network Management with YANG

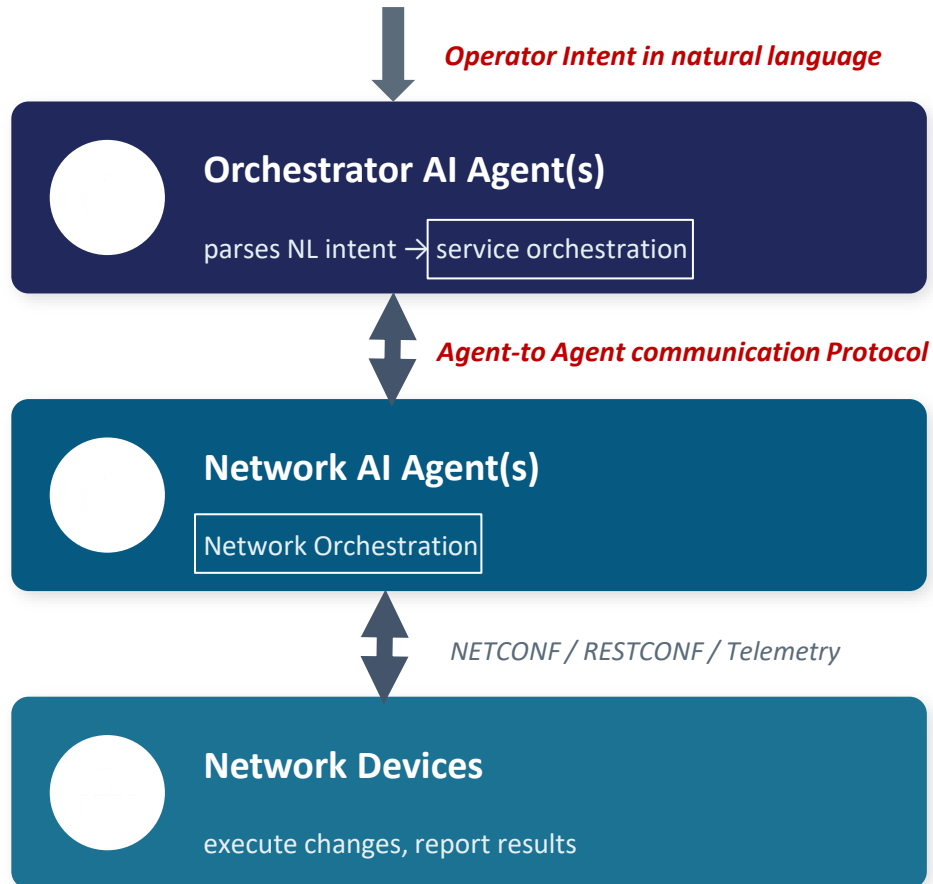


Architecture Characteristics

- Deterministic workflow
 - Automation follows rigid, predefined workflows
 - Hard-coded logic to map service->network->device layers
- Highly Structured Interfaces & Data Modeling
 - Mandates highly structured YANG data models for every layer

Shift towards an Agentic AI based Hierarchy

If the pipeline is re-implemented with AI Agents at each/some layers, what would change?



- Agent northbound interfaces are no longer YANG-driven management protocols – it now could travel alongside natural language intent, agent context, etc.
- The intent interpretation and workflow execution are done by a non-deterministic reasoning engine, and the agent autonomously decides which tools/APIs to invoke at which time

Workflow Examples

Intent-Driven Service Provisioning/Optimization



Network Troubleshooting

1

Operational intent

Operator requests the agent to continuously monitor, diagnose, and autonomously resolve issues.

2

Network operational state awareness

Agent understands the network operational state in real-time.

3

Diagnose & Plan

Once a fault is identified, the agent determines root cause (e.g., run sequences of OAM tests or perform incident-diagnose RPC) and generates a targeted repair plan.

4

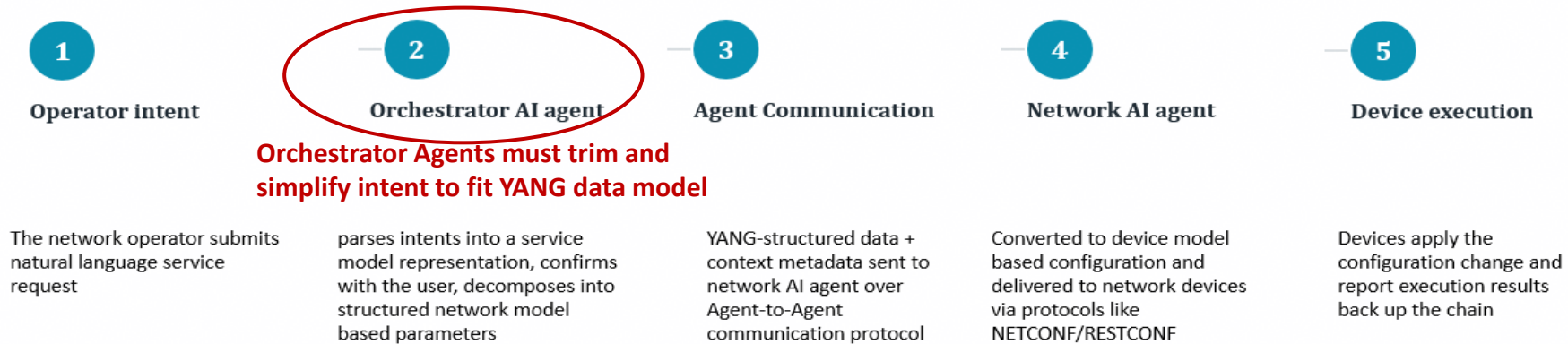
Simulate & Execute

Digital-twin simulation validates the fix, then the agent delivers YANG configuration changes to network devices.

Core Challenges Analysis

Gap Analysis 1: The Semantic Divide

Workflow Example 1: intent-driven service provisioning / optimization



Natural Language:

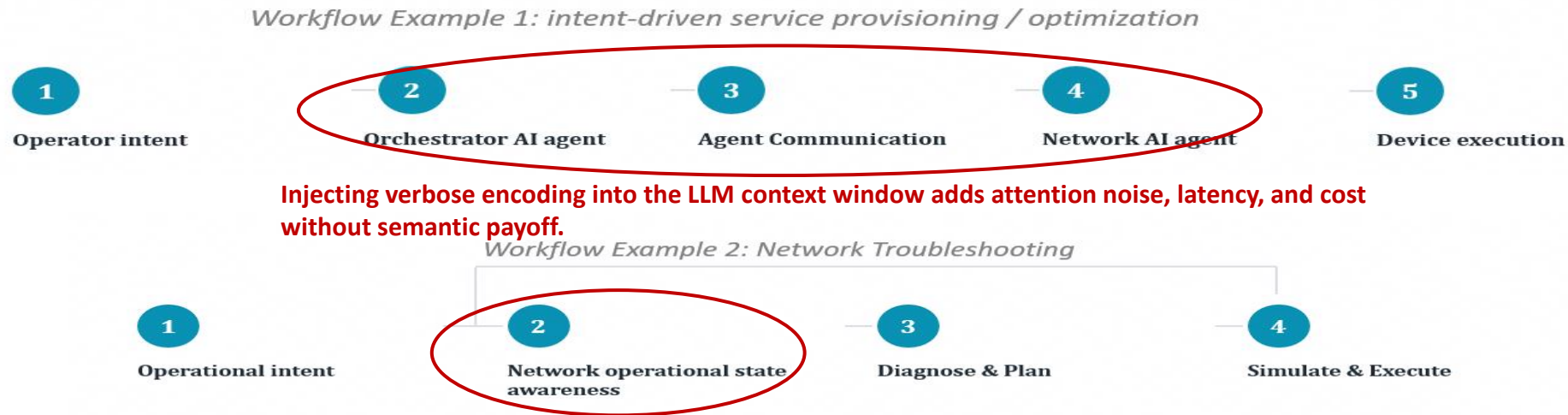
- Advantages
 - Empowers human operators to express richer intent, e.g., expectations, constraints, preferences, retry strategies, and fallback rules
 - Communication protocols like A2A natively support unstructured plain text
- Limitations
 - Could be ambiguous, and lacks the precision needed for network operations

Structured YANG:

- Advantages
 - ensure deterministic execution with highly structured and validated schema
- Limitations
 - Completely inflexible and has expressiveness limits

The core challenge is translating high-level natural language intent into precise YANG representations without losing context.

Gap Analysis 2: Token Efficiency and Encoding Overhead



Key Concepts

- LLMs don't read text as humans do – they process tokens. Tokens are the basic units of text that LLMs process.
- Each model has a maximum number of tokens it can handle (context window)
- Both input (prompt + context) and output (responses) consumes tokens
- LLM tokens cost money
- Structured formats (JSON/XML) often cost more tokens due to repeated field names/markup.
- JSON would be the preferred format compared with XML
 - The extra baggage (e.g., verbose syntax and nested tags) in XML increases processing time and computational resources.

The core challenge is utilizing verbose encoding formats such as XML and JSON for LLM reasoning and agent communication

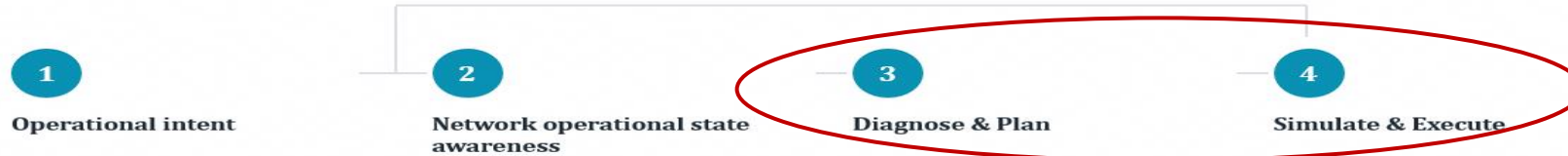
Gap Analysis 3: Non-deterministic Task Handling

Workflow Example 1: intent-driven service provisioning / optimization



The same high-level intent can produce different YANG instance data across invocations; the same observed network state can yield multiple, distinct diagnostic trajectories and conclusions.

Workflow Example 2: Network Troubleshooting



Root-causes of Agent non-determinism (Why it happens)

- **Probabilistic Nature of LLMs:** At their core, LLM runs on next-token prediction probabilities, which inherently introduces randomness.
- **Context drift:** even minor shifts in historical context can alter outcomes for identical tasks.
- **Semantic Ambiguity:** Ambiguous natural language instructions allows for multiple interpretations

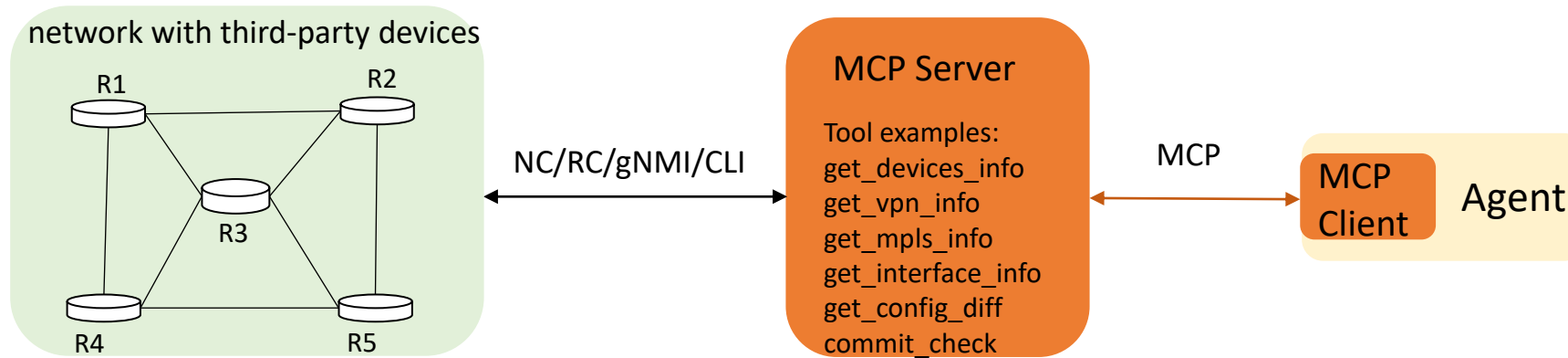
The core challenge is AI Agents introduces probability that conflicts the deterministic foundations of model-driven automation.

Key impacts of non-determinism (The Consequences)

- **Configuration Inconsistency:** divergent reasoning path can lead to inconsistent YANG config being pushed to devices
- **Operational Complexity:** Troubleshooting a failure becomes even harder if the diagnostic logic varies from run to run.
- **Operational Risk:** randomness in config generation increases the risk of misconfiguration, service impairment, etc.

Potential Ways Forward

YANG-driven Operations as AI-Invocable Tools



Core idea:

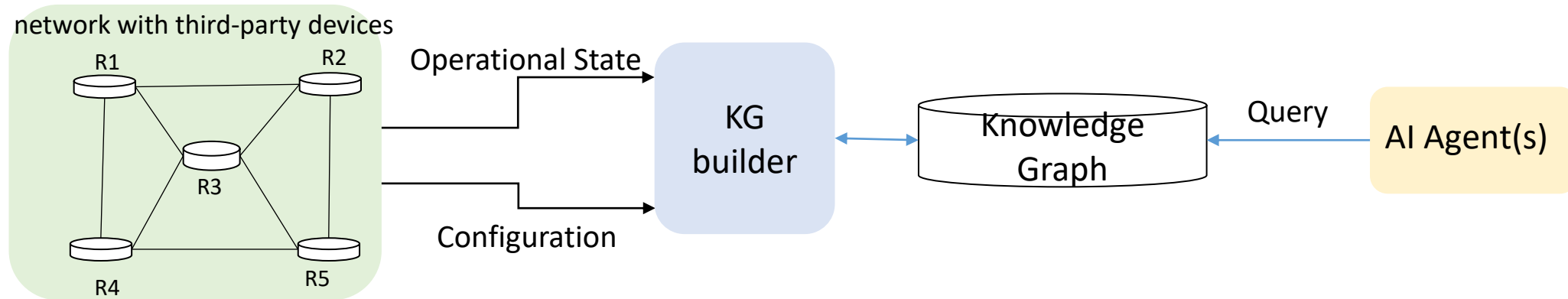
- refactor YANG-based network operations into AI-invocable tools using the Model Context Protocol (MCP)
- The abstraction layer shields agents from low-level YANG syntax and schema complexity, reducing non-determinism and hallucination risks

Industry exploration:

<https://github.com/jillesca/gNMIBuddy>

<https://github.com/Juniper/junos-mcp-server>

Semantic Enrichment via Knowledge Graphs



Core idea:

- add a semantic layer on top of YANG data models to address YANG expressiveness limits via knowledge graph, which can explicitly model concepts absent from YANG, e.g., “preferred vs. optional” and “transient vs. persistent failure”
- KGs give AI agents a machine-readable view of network semantics, relationships, and constraints. E.g., for the intent “check why the VPN tunnel is down,” a KG-enhanced agent queries relationships between the affected services and metrics

Related IETF efforts:

<https://datatracker.ietf.org/doc/draft-mackey-nmop-kg-for-netops/>

<https://datatracker.ietf.org/doc/draft-tailhardat-nmop-incident-management-noria/>

<https://datatracker.ietf.org/doc/draft-ietf-nmop-network-anomaly-semantics/>

Declarative Intent/Policy Overlay



Core idea:

- Instead of forcing agents to translate loose, high-level intent directly into rigid YANG leaf nodes, the agent first compiles the intent into a declarative intent overlay as an immediate semantic layer
- The overlay could explicitly capture some semantics like expectations, constraints, implications that existing YANG models cannot represent
- This mitigates the ambiguity and inaccuracy in natural language intent expression

Industry exploration:

- 3GPP intent meta model
- TMF intent management API
- <https://datatracker.ietf.org/doc/draft-feng-netconf-naim-op/>

Token-efficient Data Format for LLM input? TOON?

- TOON (Token-Oriented Object Notation)
 - a data serialization format designed specifically for LLM applications.
 - Zero syntax redundancy: removes closing XML tags and JSON braces
 - Context window relief: 30%~60% less token consumption
 - Think of it as a translation layer: use JSON programmatically, and encode it as TOON for LLM input.
 - Official Repositories:
 - <https://github.com/toon-format/toon>
 - <https://github.com/toon-format/spec>

Each format ranked by efficiency (accuracy percentage per 1,000 tokens):

TOON		27.7 acc%/1K tok		76.4% acc		2,759 tokens
JSON compact		23.7 acc%/1K tok		73.7% acc		3,104 tokens
YAML		19.9 acc%/1K tok		74.5% acc		3,749 tokens
JSON		16.4 acc%/1K tok		75.0% acc		4,587 tokens
XML		13.8 acc%/1K tok		72.1% acc		5,221 tokens

Efficiency score = (Accuracy % ÷ Tokens) × 1,000. Higher is better.

TOON achieves 76.4% accuracy (vs JSON's 75.0%) while using 39.9% fewer tokens. However, fit for YANG data has not yet been verified.

statistics source: <https://github.com/toon-format/toon#benchmarks>

```
{
  "users": [
    {
      "id": 1,
      "name": "Alice",
      "role": "Admin",
      "status": "Active"
    },
    {
      "id": 2,
      "name": "Bob",
      "role": "User",
      "status": "Inactive"
    },
    {
      "id": 3,
      "name": "Charlie",
      "role": "User",
      "status": "Active"
    }
  ]
}
```

JSON format

```
users[3]{id,name,role,status}:
1,Alice,Admin,Active
2,Bob,User,Inactive
3,Charlie,User,Active
```

TOON format

Format Comparison

Mapping Between the Four Paths and the Gaps

Which Solution could potentially bridge which Identified Gaps

Possible Ways Forward

Gap 1
Semantic Divide

Gap 2
Token Efficiency

Gap 3
Non-determinism

① AI-invocable Tools



② Semantic Enhancement via Knowledge Graphs & Semantic Metadata



③ Declarative Intent/Policy Overlay



④ Token-Efficient YANG Serialization



Comments, Questions, Concerns?